Software Requirements Specification: Online Complaint Management System

Oct 14, 2026 · @gauri

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Online Complaint Management System (OCMS)**, a Python web application that lets users file complaints online, track them to resolution, and lets administrators and staff handle them from one dashboard. It is the reference for design, coding, testing and evaluation of the project.

1.2 Scope

OCMS replaces paper forms, phone calls, emails and manual registers with a single database-backed system. It covers user registration and login, complaint submission with attachments, categorisation, assignment to staff, status tracking, admin responses, feedback and reporting. It does not cover payment handling, native mobile apps or integration with external ERP systems.

The system is suitable for a college, hostel, residential society, small company or local service office.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
OCMS	Online Complaint Management System
SRS	Software Requirements Specification
Complainant / User	A registered person who submits complaints
Admin	Person who manages users, categories, complaints and reports
Staff	Person assigned to resolve a complaint
FR / NFR	Functional / Non-Functional Requirement
CRUD	Create, Read, Update, Delete

Term	Meaning
ORM	Object-Relational Mapping (SQLAlchemy / Django ORM)
SLA	Service Level Agreement: target time to resolve a complaint
CSRF	Cross-Site Request Forgery

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- Project presentation: Online Complaint Management System, Primus School of Management Studies
- Python, Flask / Django and MySQL official documentation

1.5 Overview of the Document

Section 2 describes the product in general terms. Section 3 lists the functional requirements by module. Sections 4 and 5 cover external interfaces and quality attributes. Sections 6 and 7 give models and the database design, and Section 8 covers the technology stack and future scope.

2. Overall Description

2.1 Product Perspective

OCMS is a new, self-contained, three-tier web application: a browser front end, a Python back end that holds the business logic, and a relational database. It has no dependency on any existing system, and every user reaches it through a web browser.

2.2 Product Functions (Summary)

- Register, log in, reset a forgotten password and manage a profile.
- Submit complaints with category, priority, description and attachments, and receive a unique complaint number.
- Track status, view history and the full activity timeline of each complaint.
- Admin: review, assign, reprioritise, respond to and close complaints; manage users, staff and categories.
- Staff: see assigned complaints, update progress and add resolution notes.
- Email notifications, feedback and rating after closure, reports with CSV/PDF export.

- **Added for this version:** priority levels and SLA deadlines, automatic escalation of overdue complaints, complaint comments, audit log, dashboard charts, search and filters, and an anonymous-complaint option.

2.3 User Classes and Characteristics

User class	Description	Technical skill
User (Complainant)	Student, resident, employee or customer who files complaints	Basic web use
Staff	Officer or technician who resolves assigned complaints	Basic web use
Admin	Manages the whole system, users and reports	Moderate
Super Admin (optional)	Creates admins and changes system settings	Moderate

2.4 Operating Environment

- **Client:** any modern browser (Chrome, Edge, Firefox) on desktop, tablet or phone.
- **Server:** Windows or Linux with Python 3.10 or later.
- **Database:** MySQL 8 (SQLite allowed during development).
- **Email:** SMTP server (for example Gmail SMTP) for notifications.

2.5 Design and Implementation Constraints

- The back end must be written in Python using a web framework (Flask or Django).
- Passwords must be stored only as salted hashes.
- Uploaded files are limited to 5 MB each and to JPG, PNG or PDF types.
- The project must be completed within the academic semester using open-source tools only.

2.6 Assumptions and Dependencies

- Users have internet or intranet access and a valid email address.
- An SMTP account is available for sending notifications.
- The organisation supplies the list of complaint categories and staff.
- A single server instance is enough for the expected load of up to a few hundred users.

3. System Features (Functional Requirements)

Priority: **H** = must have, **M** = should have, **L** = nice to have. Requirements marked **(New)** are additions to the original project proposal.

3.1 User Management Module

ID	Requirement	Priority
FR-1.1	A visitor shall register with name, email, phone and password.	H
FR-1.2	The system shall reject duplicate emails and weak passwords (under 8 characters, no digit).	H
FR-1.3	Users, staff and admins shall log in with email and password and be redirected to their own dashboard.	H
FR-1.4	Passwords shall be stored as salted hashes (bcrypt or PBKDF2).	H
FR-1.5	A user shall view and edit their profile and change their password.	M
FR-1.6	(New) A user shall reset a forgotten password through a time-limited link sent by email.	M
FR-1.7	(New) Role-based access control shall limit each page to the allowed roles.	H
FR-1.8	(New) The account shall lock for 15 minutes after 5 failed login attempts.	M

3.2 Complaint Submission and Tracking Module

ID	Requirement	Priority
FR-2.1	A logged-in user shall submit a complaint with category, subject, description and location or department.	H
FR-2.2	The system shall generate a unique complaint number (for example OCMS-2026-0001) and show it to the user.	H

ID	Requirement	Priority
FR-2.3	The user shall attach up to 3 images or documents (JPG, PNG, PDF, max 5 MB each).	H
FR-2.4	The system shall set the initial status to Submitted and record the date and time.	H
FR-2.5	A user shall see a list of their own complaints with current status and open any complaint to see its timeline.	H
FR-2.6	A user shall search their complaints by number, keyword or status.	M
FR-2.7	(New) The user shall choose a priority (Low, Medium, High, Urgent) when submitting.	M
FR-2.8	(New) A user shall add comments to an open complaint and the admin or staff shall reply, forming a conversation thread.	M
FR-2.9	(New) A user shall withdraw a complaint that is still Submitted.	L
FR-2.10	(New) A user shall reopen a Resolved complaint within 7 days if the problem persists.	M
FR-2.11	(New) A user shall choose to file a complaint anonymously; the identity stays hidden from staff but visible to the super admin.	L

3.3 Admin Module

ID	Requirement	Priority
FR-3.1	The admin shall view all complaints in a table with search, sort and filters (status, category, priority, date range, staff).	H
FR-3.2	The admin shall open a complaint, view attachments and the user's details, and change its status.	H
FR-3.3	The admin shall assign a complaint to a staff member and reassign it later.	H

ID	Requirement	Priority
FR-3.4	The admin shall add, edit and deactivate complaint categories.	H
FR-3.5	The admin shall add, edit and deactivate users and staff accounts.	H
FR-3.6	The admin shall respond to the user with a remark that is stored in the complaint timeline.	H
FR-3.7	(New) The admin shall change a complaint's priority and set an SLA deadline per priority (for example Urgent 24 h, High 48 h, Medium 5 days, Low 10 days).	M
FR-3.8	(New) The system shall mark complaints that pass their SLA deadline as Overdue and escalate them to the admin dashboard and by email.	M
FR-3.9	(New) The admin shall merge duplicate complaints and link them to one parent complaint.	L
FR-3.10	(New) The system shall keep an audit log of admin actions (who, what, when).	M

3.4 Staff Module (New)

ID	Requirement	Priority
FR-4.1	Staff shall log in and see only complaints assigned to them.	H
FR-4.2	Staff shall update status (In Progress, Resolved) and add resolution notes.	H
FR-4.3	Staff shall mark a complaint Rejected only with a mandatory reason that the user can see.	M
FR-4.4	Staff shall see their own counts of pending, resolved and overdue complaints.	L

3.5 Notification Module (New)

ID	Requirement	Priority
FR-5.1	The system shall email the user when a complaint is submitted, assigned, updated, resolved or rejected.	M
FR-5.2	The system shall email staff when a complaint is assigned to them.	M
FR-5.3	The system shall show in-app notifications with an unread count in the navigation bar.	M
FR-5.4	Emails shall be sent in a background thread or task so the page does not wait for the mail server.	M

3.6 Feedback Module

ID	Requirement	Priority
FR-6.1	After a complaint is Resolved, the user shall give a 1 to 5 star rating and an optional comment.	H
FR-6.2	A complaint shall close automatically 7 days after resolution if the user does not reopen it.	L
FR-6.3	The admin shall view average ratings per category and per staff member.	M

3.7 Report and Dashboard Module

ID	Requirement	Priority
FR-7.1	The admin dashboard shall show totals for submitted, in-progress, resolved, rejected and overdue complaints.	H
FR-7.2	The system shall generate reports by date range, category, status and staff.	H
FR-7.3	Reports shall be exportable to CSV and PDF.	M

ID	Requirement	Priority
FR-7.4	(New) The dashboard shall show charts: complaints per category, monthly trend and average resolution time.	M
FR-7.5	(New) The system shall show the average resolution time and the percentage resolved within SLA.	M

3.8 Python-Specific Enhancements (New)

These features use Python's strengths and make the project more impressive without large extra effort.

ID	Requirement	Priority
FR-8.1	Keyword-based auto-categorisation: when the user types a description, the system suggests a category (for example "wifi", "router" suggests Network).	L
FR-8.2	Duplicate detection: when a user types a subject, the system shows similar open complaints using text similarity (scikit-learn TF-IDF or difflib).	L
FR-8.3	Simple sentiment check on the description (TextBlob or VADER) to flag very negative complaints for faster review.	L
FR-8.4	A scheduled job (APScheduler or Celery beat) shall run hourly to mark overdue complaints and auto-close resolved ones.	M
FR-8.5	A REST API (Flask-RESTful or Django REST Framework) shall expose complaint data in JSON for future mobile apps.	L

4. External Interface Requirements

4.1 User Interfaces

- A responsive layout built with HTML, CSS, JavaScript and Bootstrap that works on screens from 360 px wide upward.

- Pages: Home, Register, Login, Forgot Password, User Dashboard, Submit Complaint, My Complaints, Complaint Details, Profile, Admin Dashboard, Complaint List, Assign Complaint, Manage Users, Manage Categories, Reports, Staff Dashboard.
- Every form shall show clear validation messages, and status shall be shown as a coloured badge (Submitted: grey, In Progress: blue, Resolved: green, Rejected: red, Overdue: orange).
- Navigation shall be consistent, with the logged-in user's name and a Logout link in the header.

4.2 Hardware Interfaces

- No special hardware is needed. A standard PC, laptop, tablet or smartphone with a browser can use the system.
- The server needs at least 2 GB RAM, 1 GB free disk space (more for attachments) and a network connection.

4.3 Software Interfaces

Component	Interface
Python 3.10+	Runtime for the back end
Flask or Django	Web framework: routing, templates, sessions
MySQL 8 via SQLAlchemy / Django ORM	Persistent storage of all data
SMTP server	Outgoing email notifications
ReportLab or WeasyPrint	PDF report generation
Chart.js	Dashboard charts in the browser
Web browser	Chrome, Edge or Firefox, current versions

4.4 Communication Interfaces

- HTTP/HTTPS between browser and server; HTTPS shall be used in deployment.
- SMTP (port 587, TLS) for email.
- JSON over HTTP for the optional REST API.

5. Non-Functional Requirements

ID	Attribute	Requirement
NFR-1	Security	Passwords are hashed with salt; sessions expire after 30 minutes of inactivity; all forms carry CSRF tokens.
NFR-2	Security	All database access uses the ORM or parameterised queries to prevent SQL injection; output is escaped to prevent XSS.
NFR-3	Security	Uploaded files are checked for type and size, renamed with a random name and stored outside the public web root.
NFR-4	Security	A user can see only their own complaints; staff can see only assigned complaints; admin pages reject other roles with a 403 error.
NFR-5	Performance	Common pages load within 3 seconds and list pages show 10 to 20 rows per page (pagination) with up to 50 concurrent users.
NFR-6	Usability	A new user shall be able to register and file a complaint in under 3 minutes without training.
NFR-7	Reliability	Complaint data is saved in a database transaction, so a failure never leaves half-saved records; a daily database backup is taken.
NFR-8	Availability	The system should be available at least 95% of the time during working hours.
NFR-9	Scalability	The design is modular so new modules, such as SMS, can be added, and the database can later move to a larger server.
NFR-10	Maintainability	Code follows PEP 8, is split into modules (models, views, services, templates), carries docstrings and has configuration in environment variables.
NFR-11	Portability	The system runs on Windows and Linux with the same code.
NFR-12	Compatibility	The interface works on the current versions of Chrome, Edge and Firefox and on mobile browsers.

ID	Attribute	Requirement
NFR-13	Testability	Core functions have unit tests (pytest or unittest) with at least 60% coverage of the service layer.

6. System Models

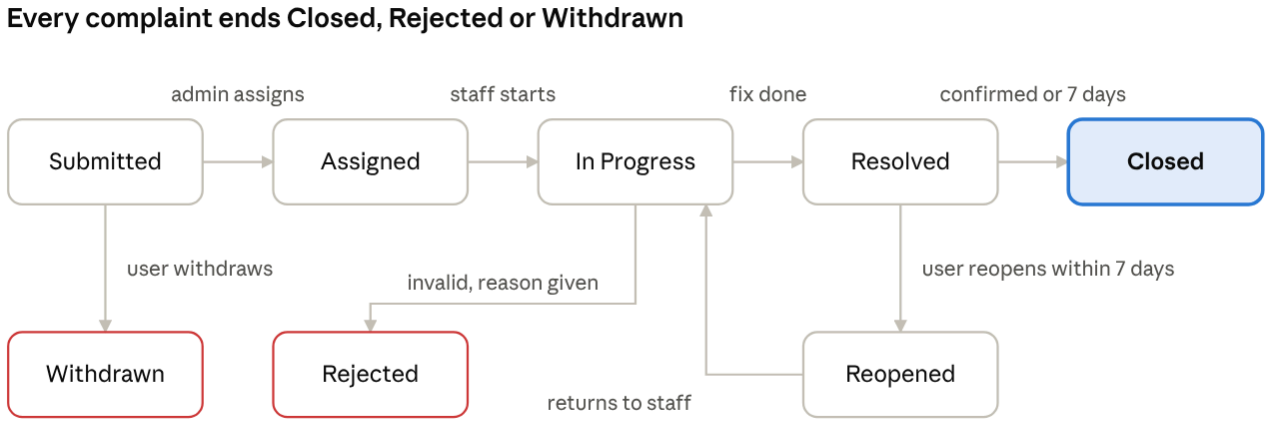
6.1 Use Cases

Actor	Use cases
Visitor	Register, log in, reset password
User	Submit complaint, attach files, track status, comment, withdraw, reopen, give feedback, edit profile
Staff	View assigned complaints, update status, add resolution notes, reject with reason
Admin	Review and assign complaints, change priority, respond, manage users and categories, view reports and audit log
System (scheduler)	Send emails, mark overdue complaints, auto-close resolved complaints

6.2 Complaint Workflow

1. The user registers or logs in.
2. The user submits a complaint and receives a complaint number.
3. The system stores it with status Submitted and notifies the admin.
4. The admin reviews it, sets the priority and assigns a staff member.
5. Staff work on it and update the status, adding notes the user can read.
6. The user tracks progress and can comment at any time.
7. Staff mark it Resolved and the user is asked for a rating and feedback.
8. The complaint becomes Closed when the user confirms or after 7 days.

6.3 Complaint Status Lifecycle



complaint status lifecycle · 8 states

A complaint normally runs left to right from Submitted to Closed. Withdrawn is only possible before assignment, Rejected needs a written reason, and Reopened returns the complaint to staff. Overdue is a flag set by the scheduler when the SLA deadline passes, not a separate status.

7. Database Design

OCMS uses a normalised MySQL schema (third normal form). PK = primary key, FK = foreign key. The original proposal's separate Admin table is merged into a single Users table with a role column, which removes duplicate login logic.

7.1 Tables

users

Field	Type	Notes
user_id	INT	PK, auto increment
name	VARCHAR(100)	Not null
email	VARCHAR(120)	Unique, not null
phone	VARCHAR(15)	
password_hash	VARCHAR(255)	Not null
role	ENUM	user, staff, admin, superadmin
is_active	BOOLEAN	Default true

Field	Type	Notes
failed_attempts, locked_until	INT, DATETIME	For account lockout
created_at	DATETIME	

categories

Field	Type	Notes
category_id	INT	PK
name	VARCHAR(80)	Unique
description	TEXT	
is_active	BOOLEAN	

complaints

Field	Type	Notes
complaint_id	INT	PK
complaint_no	VARCHAR(20)	Unique, such as OCMS-2026-0001
user_id	INT	FK to users
category_id	INT	FK to categories
assigned_to	INT	FK to users (staff), nullable
subject	VARCHAR(150)	
description	TEXT	
priority	ENUM	Low, Medium, High, Urgent
status	ENUM	Submitted, Assigned, In Progress, Resolved, Closed, Rejected, Withdrawn, Reopened
is_anonymous	BOOLEAN	
parent_id	INT	FK to complaints (for merged duplicates), nullable
created_at, due_at, resolved_at	DATETIME	due_at comes from the SLA

attachments

Field	Type	Notes
attachment_id	INT	PK
complaint_id	INT	FK to complaints
file_name, stored_name	VARCHAR	Original and random stored names
file_type, file_size	VARCHAR, INT	
uploaded_at	DATETIME	

comments (complaint timeline and conversation)

Field	Type	Notes
comment_id	INT	PK
complaint_id	INT	FK to complaints
user_id	INT	FK to users
message	TEXT	
is_status_change	BOOLEAN	True for automatic timeline entries
created_at	DATETIME	

feedback

Field	Type	Notes
feedback_id	INT	PK
complaint_id	INT	FK to complaints, unique
rating	TINYINT	1 to 5
comments	TEXT	
created_at	DATETIME	

notifications

Field	Type	Notes
notification_id	INT	PK
user_id	INT	FK to users
complaint_id	INT	FK to complaints
message	VARCHAR(255)	
is_read	BOOLEAN	
created_at	DATETIME	

audit_log

Field	Type	Notes
log_id	INT	PK
user_id	INT	FK to users (who acted)
action	VARCHAR(100)	For example "Assigned complaint"
target_id	INT	Affected record
created_at	DATETIME	

7.2 Relationships

- One user files many complaints (1:N); one category groups many complaints (1:N).
- One staff user is assigned many complaints (1:N).
- One complaint has many attachments and many comments (1:N each), and at most one feedback (1:1).
- One user receives many notifications (1:N).

8. Technology Stack, Future Scope and Appendices

8.1 Technology Stack

The original proposal listed "Python / PHP / Java". This version fixes the back end to Python.

Layer	Technology
Front end	HTML5, CSS3, JavaScript, Bootstrap 5, Jinja2 templates, Chart.js
Back end	Python 3.10+ with **Flask** (simpler, good for a first project) or **Django** (built-in admin, auth and ORM)
Database	MySQL 8 (SQLite for development)
ORM	SQLAlchemy (Flask) or Django ORM
Libraries	Flask-Login, Flask-WTF, Flask-Mail, bcrypt, APScheduler, ReportLab, pytest
Tools	Visual Studio Code, Git and GitHub, MySQL Workbench, Postman
Server	Flask development server for demo, Gunicorn with Nginx or Waitress for deployment

8.2 Suggested Project Structure (Flask)

```
ocms/
  app/
    __init__.py        # app factory
    models.py          # database tables
    auth/              # login, register, reset
    complaints/        # user complaint routes
    admin/             # admin and staff routes
    services/          # email, SLA, reports
    templates/         # Jinja2 pages
    static/            # css, js, uploads
  tests/
  config.py
  run.py
  requirements.txt
```

8.3 Future Scope

- Mobile app using the REST API; SMS and push notifications.
- Machine-learning categorisation and priority prediction trained on past complaints.
- Chatbot that answers common questions and collects complaint details.
- Multilingual support and voice-based complaint entry.
- Advanced analytics, heat maps by location and department performance scores.

- Single sign-on with college or company accounts.

8.4 Appendix A: Complaint Status Definitions

Status	Meaning
Submitted	Filed by the user, waiting for admin review
Assigned	Admin has assigned it to a staff member
In Progress	Staff is working on it
Resolved	Staff has fixed it; waiting for user feedback
Closed	User confirmed, or 7 days passed after resolution
Rejected	Invalid or out of scope, with a reason given
Withdrawn	Cancelled by the user before assignment
Reopened	User reported the problem again after Resolved

8.5 Appendix B: Suggested Development Plan

Phase	Work	Duration
1	Requirements, SRS and database design	2 weeks
2	Authentication, roles and user module	2 weeks
3	Complaint submission, attachments and tracking	3 weeks
4	Admin and staff modules, assignment, comments	3 weeks
5	Notifications, SLA scheduler, feedback	2 weeks
6	Reports, dashboard charts and exports	2 weeks
7	Testing, bug fixing, documentation and viva preparation	2 weeks

8.6 Appendix C: Sample Test Cases

ID	Test	Expected result
TC-1	Register with an existing email	Error shown; no new account

ID	Test	Expected result
TC-2	Login with a wrong password 5 times	Account locks for 15 minutes
TC-3	Submit a complaint with a 10 MB file	Rejected with a size message
TC-4	User opens another user's complaint URL	403 Forbidden
TC-5	Admin assigns a complaint	Staff sees it; user gets an email and the status changes to Assigned
TC-6	Complaint passes its SLA deadline	Marked Overdue and shown on the admin dashboard
TC-7	User rates a resolved complaint	Feedback saved; averages updated in reports